Deploying an Angular Application Using Docker and Docker Compose

Development container with Angular CLI and production deployment with Nginx

Project	Project 2: Angular Docker Deployment
Submitted by	Kashyup Gaud
Course / Subject	DevOps Lab
Date	22 July 2026

Development Dockerfile

```
FROM node:22.22.3

WORKDIR /app
COPY package*.json ./
RUN npm install

COPY . .
EXPOSE 4200
CMD ["npm", "start"]
```

The development image installs dependencies, copies the project, exposes Angular's development port, and executes the project's start script.

Production Dockerfile

```
# Build stage
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build -- --configuration production

# Production stage
FROM nginx:alpine
COPY --from=build /app/dist/angular-docker/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

The build tools remain in the temporary Node.js stage. Only Nginx and the compiled static files are retained in the final runtime image.

Development Compose File

```
services:
  angular-app:
    build:
      context: .
      dockerfile: Dockerfile.dev
    ports:
      - "4200:4200"
    environment:
      - NG_ALLOWED_HOSTS=localhost
    volumes:
      - .:/app
      - /app/node_modules
```

Production Compose File

```
services:

 angular-prod:

   build:
     context: .
     dockerfile: Dockerfile

   ports:
     - "80:80"
```

Implementation Evidence and Screenshot Analysis

The following figures document the configuration, execution, validation, and shutdown of the project. Each screenshot is paired with an explanation of the evidence it provides.

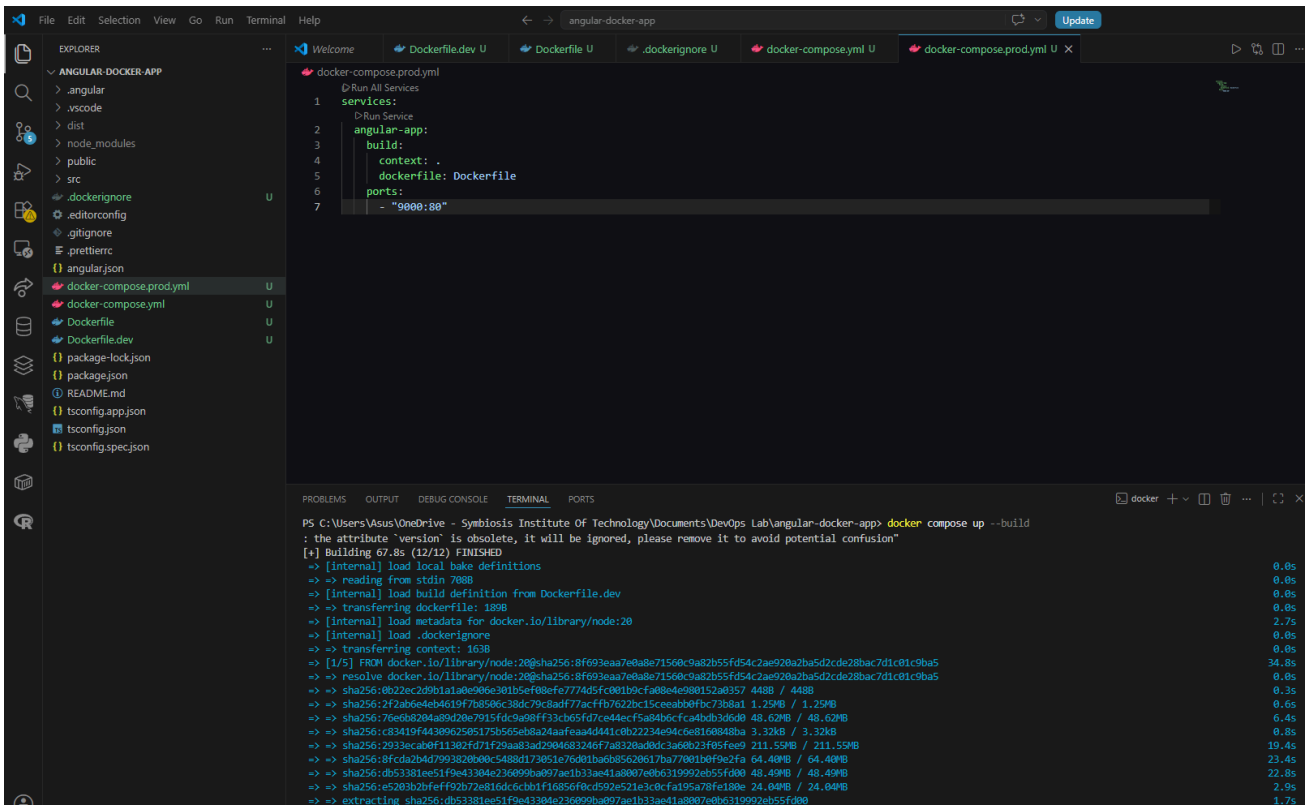

Figure 2. Angular project structure in VS Code

Explanation: The Explorer shows the Angular source folders and all required Docker configuration files: Dockerfile, Dockerfile.dev, docker-compose.yml, docker-compose.prod.yml, nginx.conf, package files, and TypeScript configuration files.

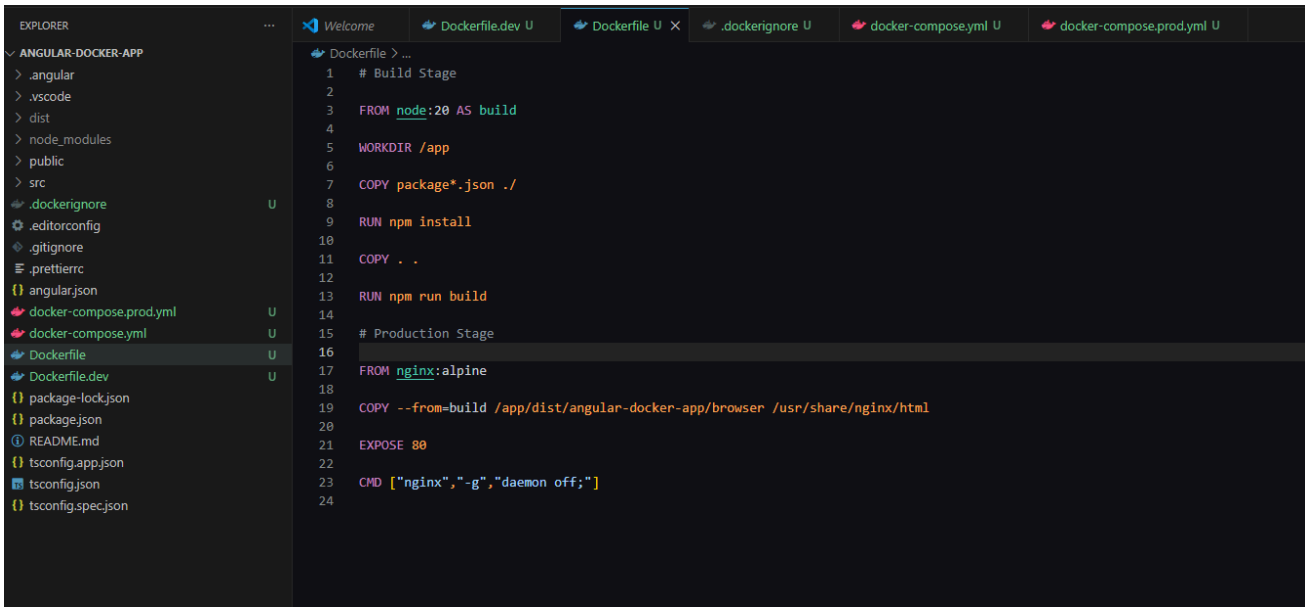

Figure 3. Development Dockerfile

Explanation: The development Dockerfile uses Node.js, sets /app as the working directory, installs npm dependencies, copies the application, exposes port 4200, and starts the Angular application.

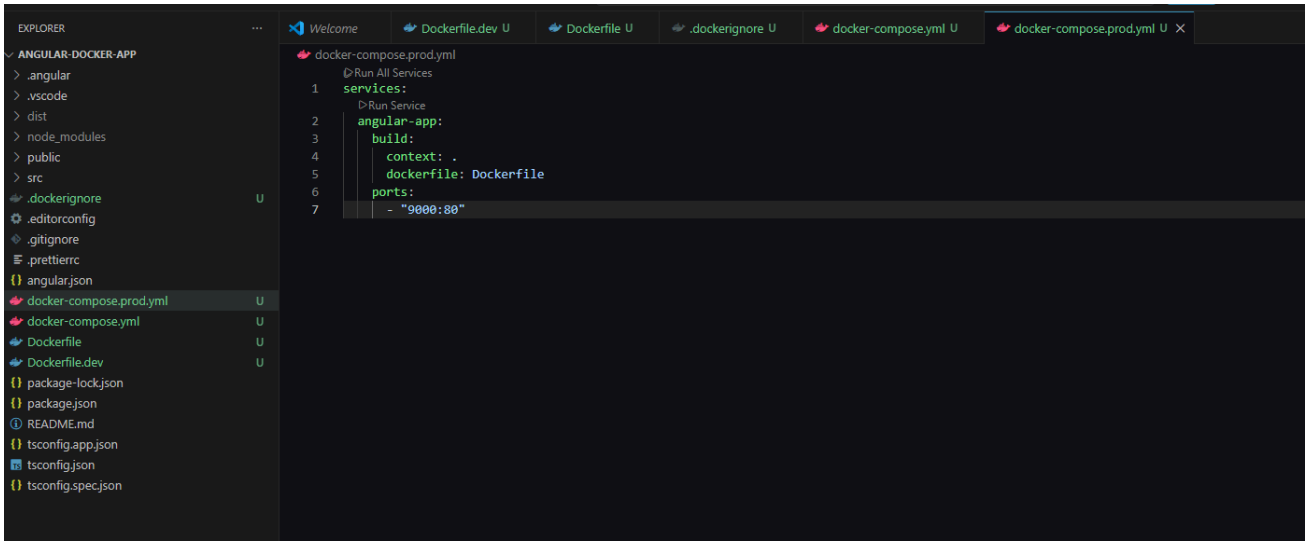

Figure 4. Multi-stage production Dockerfile

Explanation: The first stage builds the Angular application with Node.js. The second stage uses Nginx Alpine, copies the browser build into the Nginx document root, exposes port 80, and keeps Nginx in the foreground.

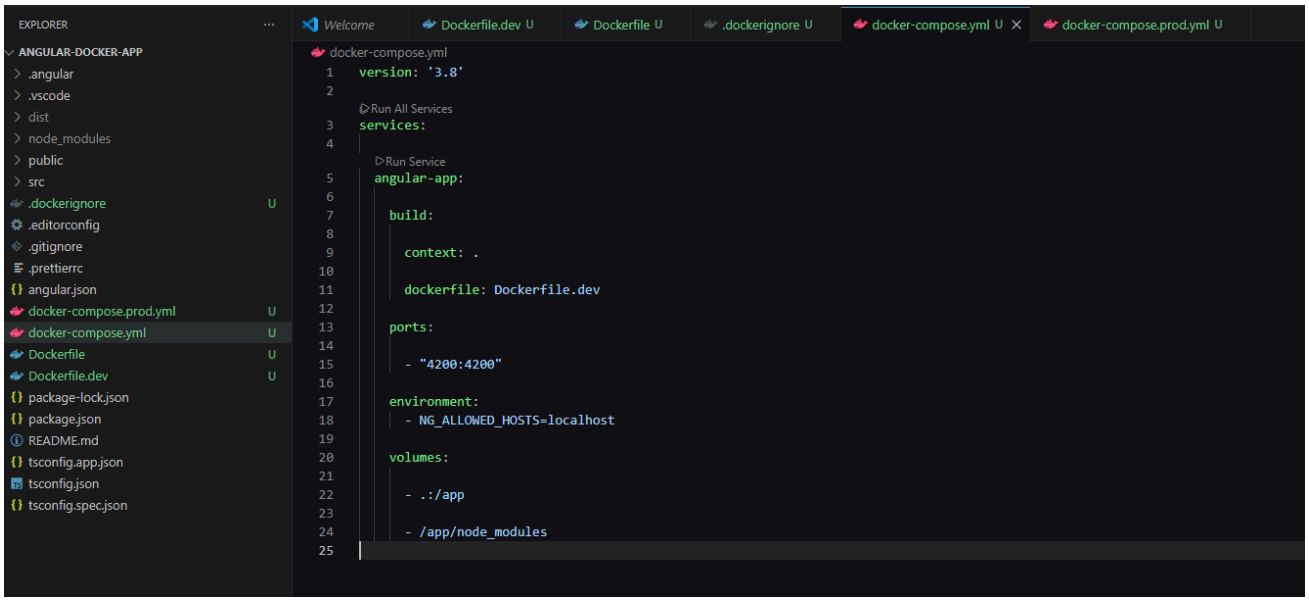

Figure 5. Development Docker Compose configuration

Explanation: The development service builds from Dockerfile.dev, maps host port 4200 to container port 4200, passes the allowed-host setting, mounts the project directory, and protects container-installed node_modules with an anonymous volume.

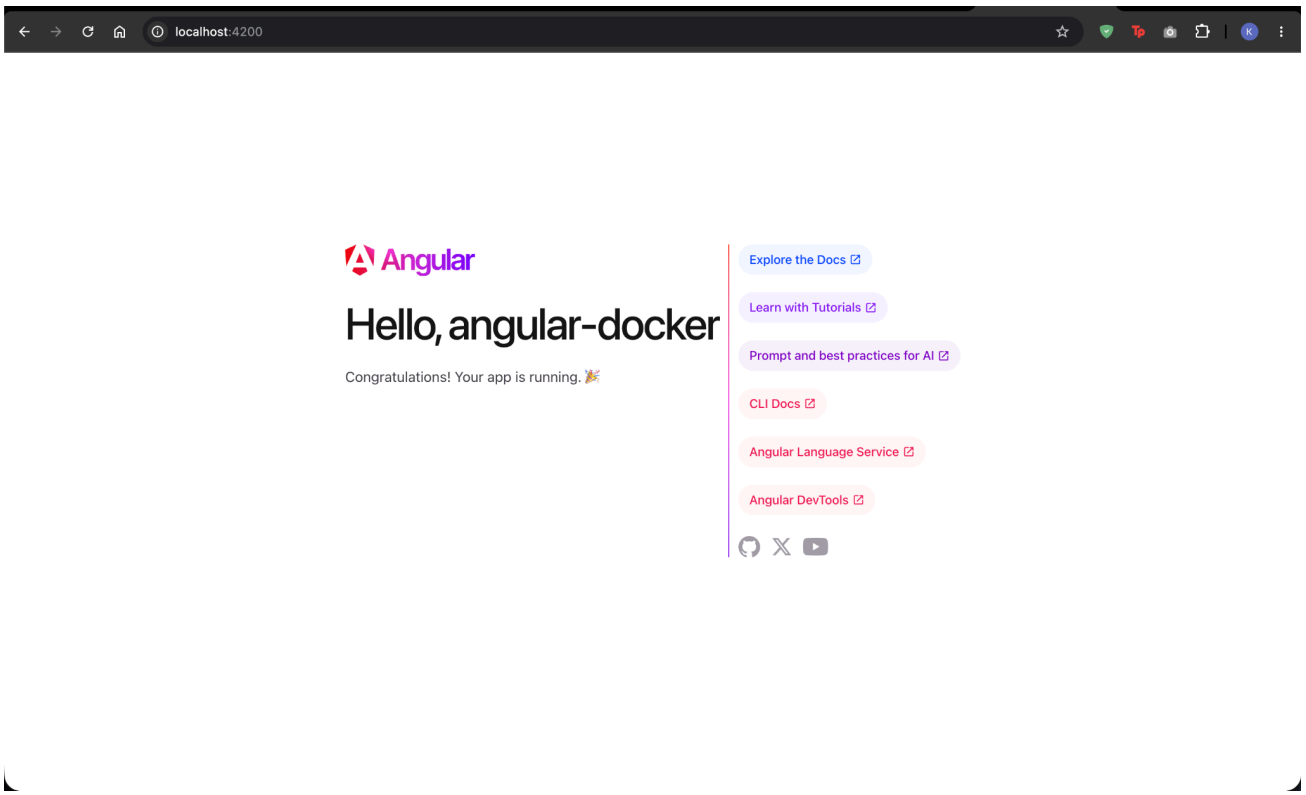

Figure 7. Angular development application on port 4200

Explanation: The browser successfully loads the Angular starter page from localhost:4200, proving that the development container is running and the port mapping is correct.

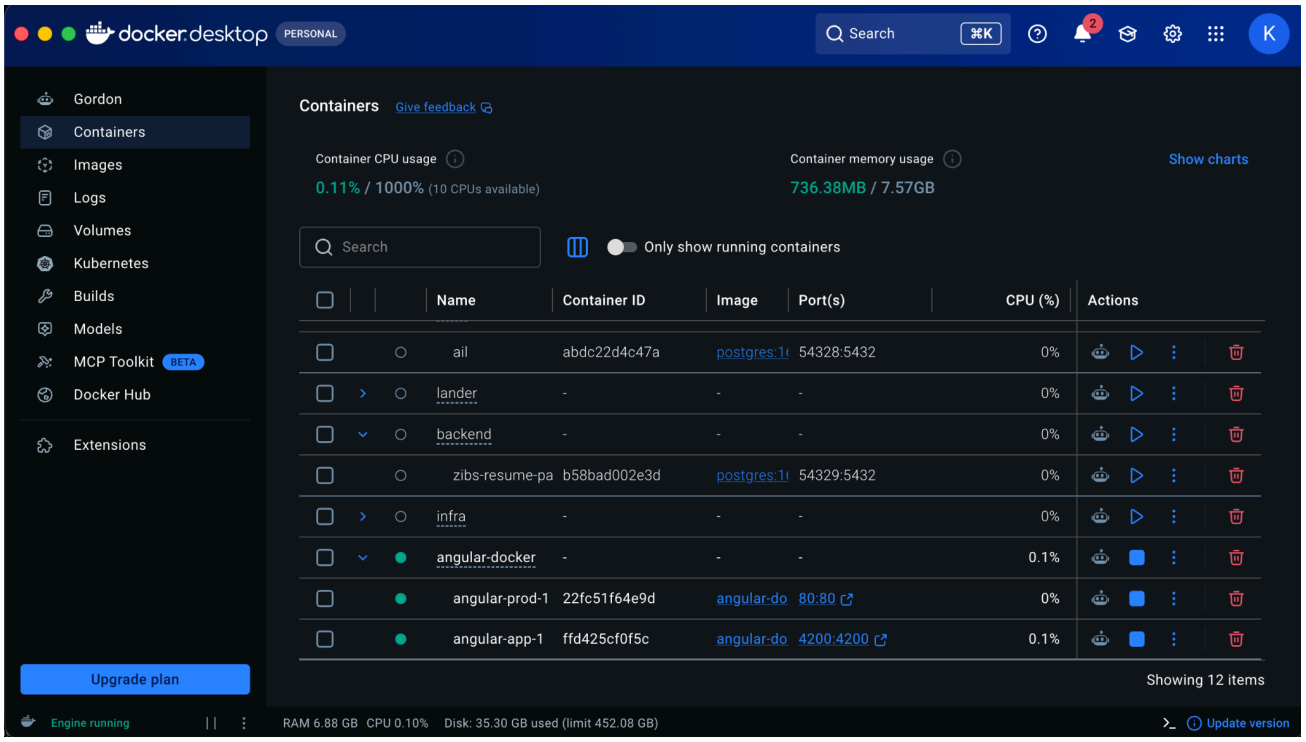

Figure 8. Running Angular containers in Docker Desktop

Explanation: Docker Desktop shows the angular-docker Compose project and its running development and production containers. The published ports 4200:4200 and 80:80 are visible.

```
kisna@MacBook-Air-2 angular-docker % docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED          STATUS          PORTS
          NAMES
fbd8c6b3c2c8   angular-app                    "/docker-entrypoint.…"   45 seconds ago   Up 45 seconds   0.0.0.0:8080->80/tcp, [::]:8080->
80/tcp    hardcore_jones
22fc51f64e9d   angular-docker-angular-prod    "/docker-entrypoint.…"   3 minutes ago    Up 3 minutes    0.0.0.0:80->80/tcp, [::]:80->80/t
cp        angular-docker-angular-prod-1
kisna@MacBook-Air-2 angular-docker %
```

Figure 9. Running containers verified with docker ps

Explanation: The terminal lists the running Angular images, container status, and host-to-container port mappings. This confirms that Docker has published the required ports.

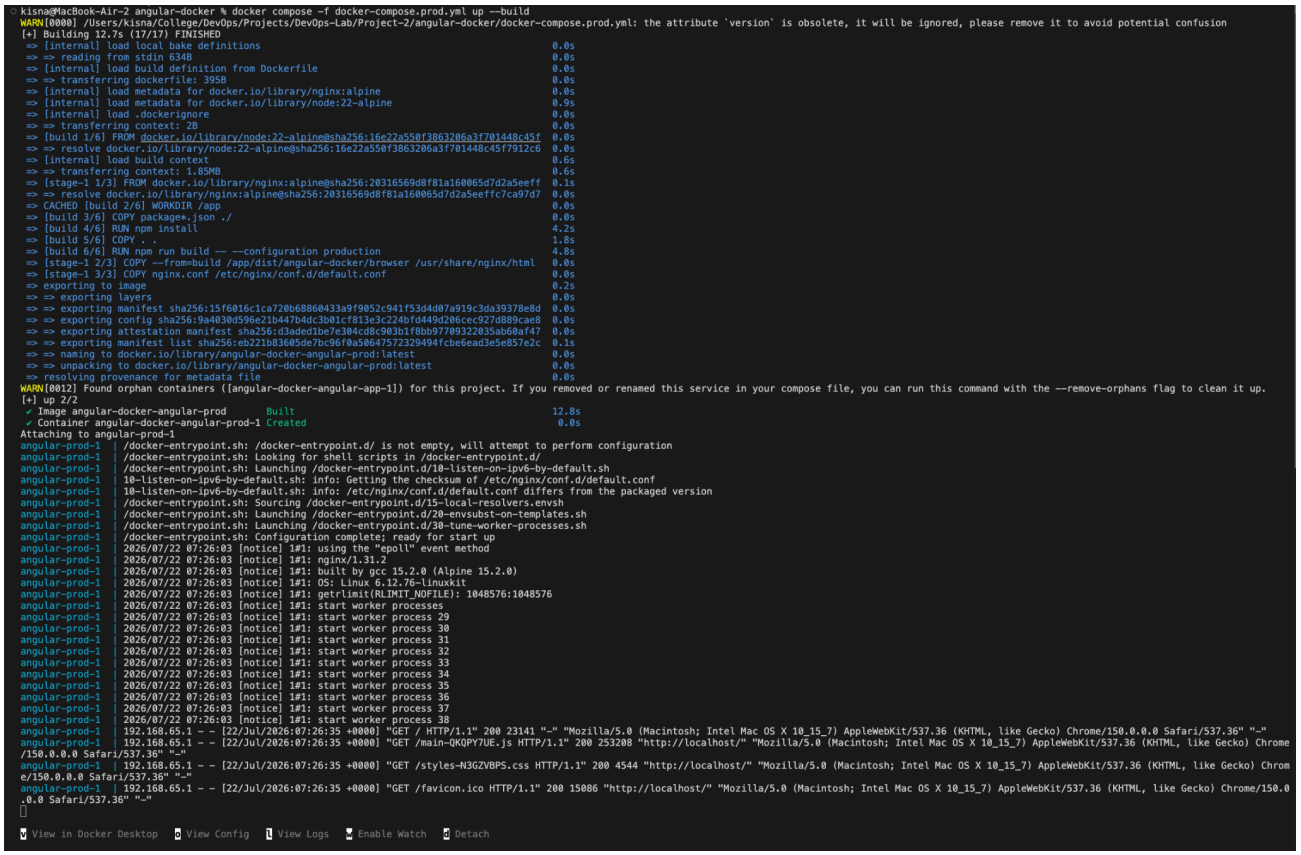

Figure 10. Production image build and Nginx startup

Explanation: The production Compose command completes the Node.js build stage, copies the Angular browser output into Nginx, creates the container, starts Nginx workers, and records successful HTTP 200 responses.

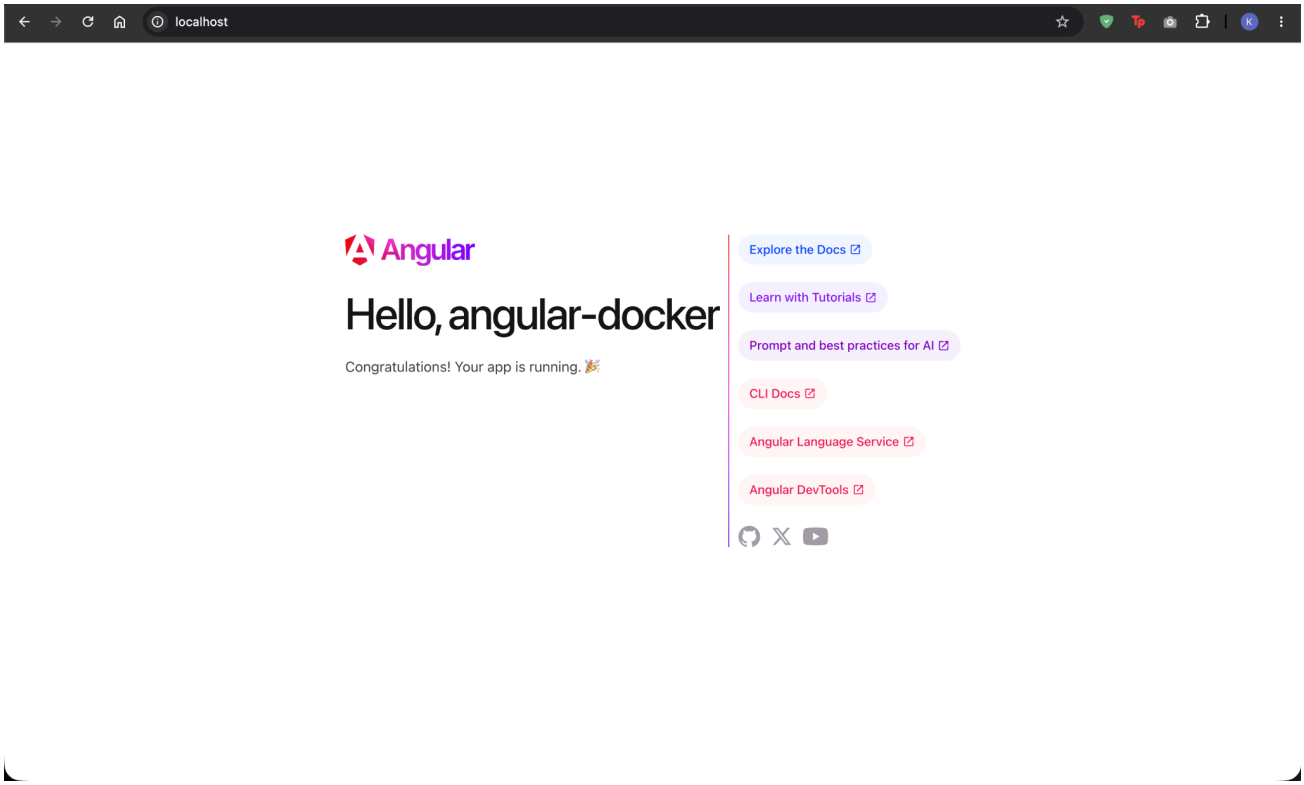

Figure 11. Production Angular application served by Nginx

Explanation: The Angular application loads at http://localhost without an explicit port because the production service is published on the standard HTTP port 80.

```
C:\Users\Asus\OneDrive - Symbiosis Institute Of Technology\Documents\DevOps Lab\angular-docker-app>docker images
                                                                                                      i Info →  U In Use
IMAGE                                                ID             DISK USAGE   CONTENT SIZE   EXTRA
ail-backend:latest                                   d04d1fb690de       3.21GB          875MB    U
ail-frontend:latest                                  a196962fcf1a        857MB          196MB    U
ail-kafka-consumer:latest                            60fc2e16b49c        411MB         94.4MB    U
ail-kafka-producer:latest                            1f17dbfe7326        411MB         94.4MB    U
angular-docker-angular-app:latest                    eaffded207eb       1.25GB          275MB    U
angular-docker-app-angular-app:latest                67e34474b4af       93.1MB         26.2MB    U
ankane/pgvector:latest                               956744bd14e9        628MB          160MB    U
confluentinc/cp-kafka:7.5.0                          fbbb6fa11b25       1.33GB          439MB    U
confluentinc/cp-kafka:latest                         68cf7eb0d994        865MB          299MB
confluentinc/cp-zookeeper:7.5.0                      02f6c042bb9a       1.33GB          439MB    U
confluentinc/cp-zookeeper:latest                     7610a50b13e7       1.75GB          593MB
hello-world:latest                                   d4aaab6242e0       25.9kB         9.52kB    U
my-flask-app:latest                                  f74cf038b7e1        200MB         48.8MB    U
pbl-2-neuro-adaptive-ai-assistant-backend:latest     7fd27bc761b2        1.1GB          245MB    U
pbl-2-neuro-adaptive-ai-assistant-frontend:latest    de1b0bc6e2d7        1.5GB          401MB    U
postgres:16                                          2586e2a95d1c        641MB          166MB    U
subtrack-backend:latest                              ec0b2ad6a724       1.13GB          272MB    U
subtrack-frontend:latest                             5f2af9eee49e       93.8MB         26.3MB    U
```

Figure 12. Docker images available on the host

Explanation: The docker images command lists the created Angular development and production images alongside the base Node.js and Nginx images used by the project.

```
● kisna@MacBook-Air-2 angular-docker % docker compose ps
  NAME                          IMAGE                         COMMAND                  SERVICE        CREATED          STATUS          PORTS
  angular-docker-angular-app-1  angular-docker-angular-app    "docker-entrypoint.s…"   angular-app    10 minutes ago   Up 10 minutes   0.0.0.0:4200->4200/tcp, [::]:4200->4200/tcp
  angular-docker-angular-prod-1 angular-docker-angular-prod   "/docker-entrypoint.…"   angular-prod   15 minutes ago   Up 15 minutes   0.0.0.0:80->80/tcp, [::]:80->80/tcp
○ kisna@MacBook-Air-2 angular-docker %
```

Figure 13. Docker Compose service status

Explanation: docker compose ps displays both Compose services, their images, running state, creation time, and published ports.

```
● kisna@MacBook-Air-2 angular-docker % docker compose down
  [+] down 2/2
   ✔ Container angular-docker-angular-app-1 Removed                                                            0.7s
   ! Network angular-docker_default         Resource is still in use                                           0.0s
● kisna@MacBook-Air-2 angular-docker % docker compose -f docker-compose.prod.yml down
  WARN[0000] /Users/kisna/College/DevOps/Projects/DevOps-Lab/Project-2/angular-docker/docker-compose.prod.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
  [+] down 2/2
   ✔ Container angular-docker-angular-prod-1 Removed                                                           0.2s
   ✔ Network angular-docker_default          Removed                                                           0.2s
○ kisna@MacBook-Air-2 angular-docker %
```

Figure 14. Stopping development and production services

Explanation: The development container is removed first, but the shared project network remains while the production container is attached. After the production Compose configuration is stopped, the remaining container and network are removed.

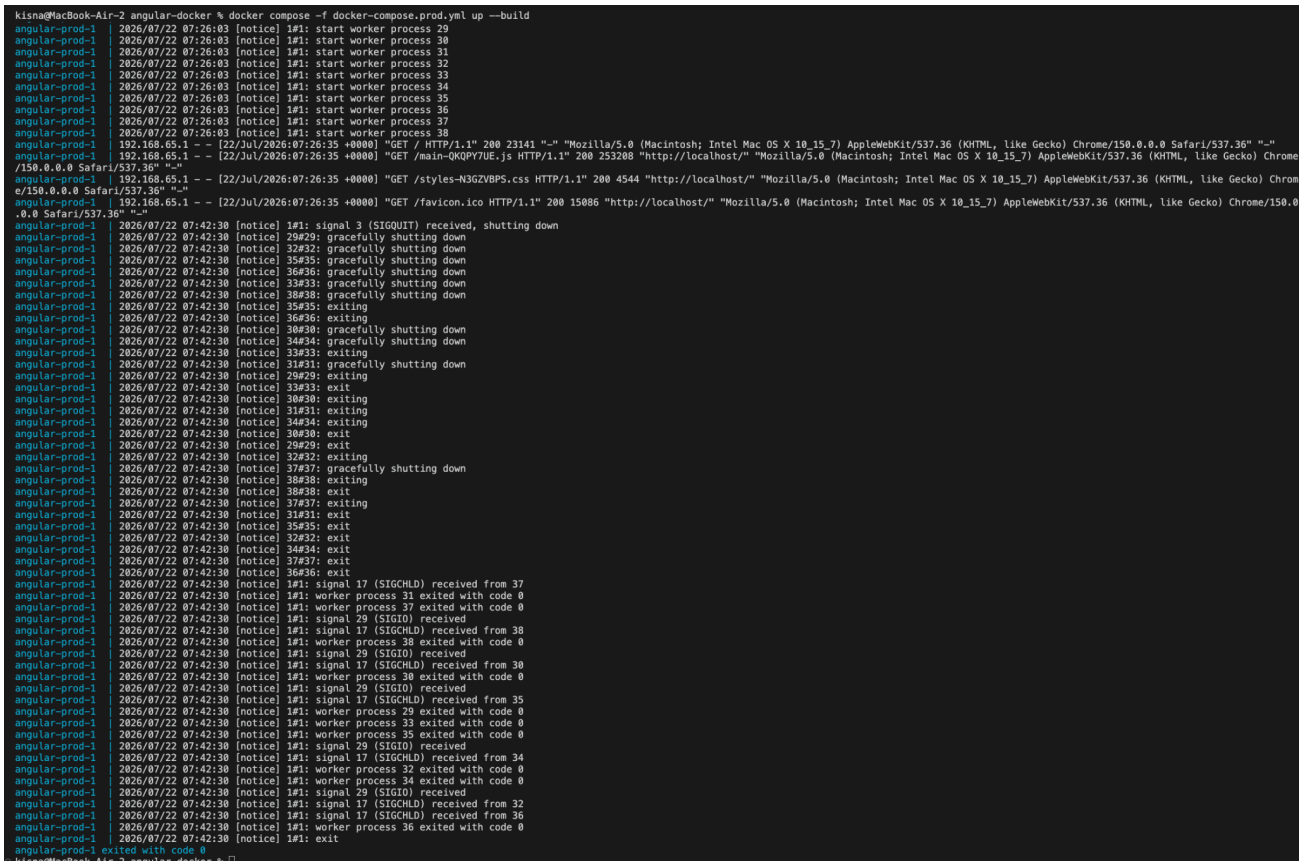

Figure 15. Graceful Nginx shutdown logs

Explanation: After the stop command, Nginx receives termination signals, gracefully shuts down its worker processes, and exits with code 0. This is normal container lifecycle behavior.

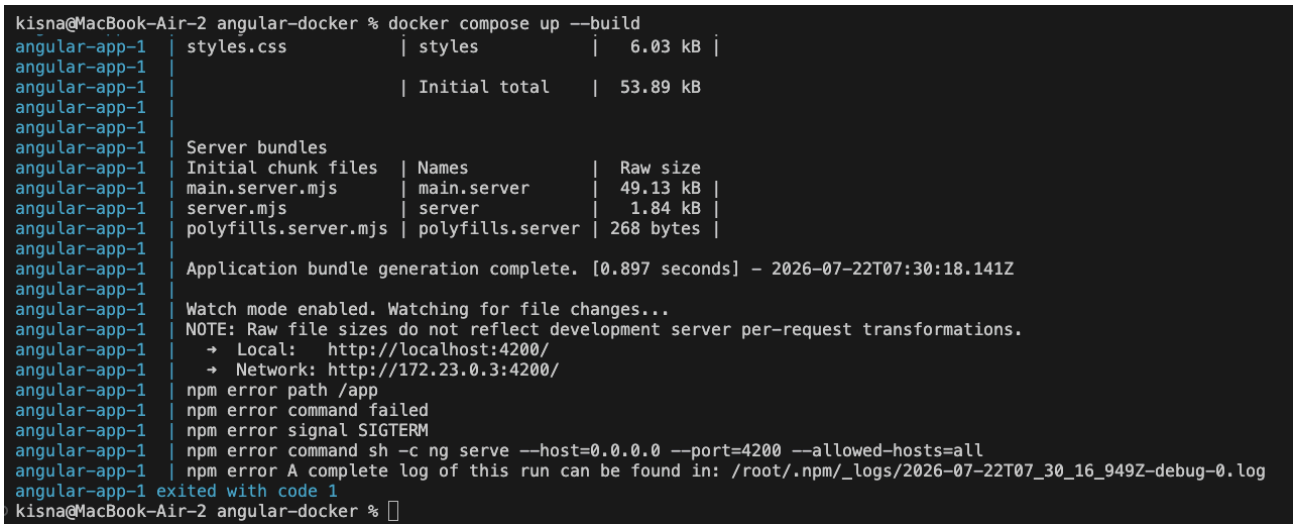

Figure 16. Angular development process terminated during shutdown

Explanation: The Angular development process reports SIGTERM and exits after the container is stopped. The message represents an intentional termination signal rather than an Angular compilation failure.

Docker Commands Explained

The project uses the following commands in sequence. Each command is shown with its practical purpose and expected behavior.

`ng new angular-docker-app`

Creates a new Angular workspace and application, generates the source structure, and installs the initial npm dependencies.

`docker compose up --build`

Reads docker-compose.yml, rebuilds the development image when required, creates the service container and network, starts Angular, and attaches the terminal to the service logs.

`docker compose down`

Stops and removes the development service containers and removes the Compose network when no other containers are still using it.

`docker compose -f docker-compose.prod.yml up --build`

Uses the production Compose file, performs the multi-stage Docker build, creates the Nginx-based service, publishes port 80, starts the container, and attaches to logs.

`docker compose -f docker-compose.prod.yml down`

Stops and removes the production service and its Compose-managed resources.

`docker ps`

Lists running containers with container IDs, images, commands, uptime, published ports, and names.

`docker images`

Lists locally available images, image identifiers, disk usage, content size, and whether an image is currently in use.

`docker compose ps`

Shows the status of services that belong to the current Compose project, including service name, state, and published ports.

`Ctrl + C`

Sends an interrupt to the foreground Compose process. Docker forwards a termination signal to the container process so it can shut down gracefully.

`docker compose up --build --remove-orphans`

Optional cleanup form that starts the selected Compose services and removes containers created by older service definitions that are no longer present.

Conclusion

The project successfully demonstrates a complete Angular containerization workflow. Development uses Node.js, Angular's live development server, source mounting, and port 4200. Production uses a multi-stage build to keep compilation dependencies out of the runtime image and serves the final static application through Nginx on port 80. Docker Compose makes both workflows repeatable and provides consistent lifecycle management. The collected evidence confirms successful builds, correct networking, browser accessibility, image creation, service status, and controlled shutdown.